

# Challenge Write-Up Template

## Challenge Information

|                     |                                  |
|---------------------|----------------------------------|
| Student Name        | Daniella Marie A. Leonud         |
| Section / Class     | BSIT-3G                          |
| Challenge Name      | bytemancy 1                      |
| Category            | General Skills                   |
| Points / Difficulty | Easy                             |
| Date Solved         | August 7, 2026                   |
| Flag Format Given   | picoCTF{h0w_m4ny_e's??_7dbc095c} |

## 1. Challenge Description

*Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copy-paste the original prompt.*

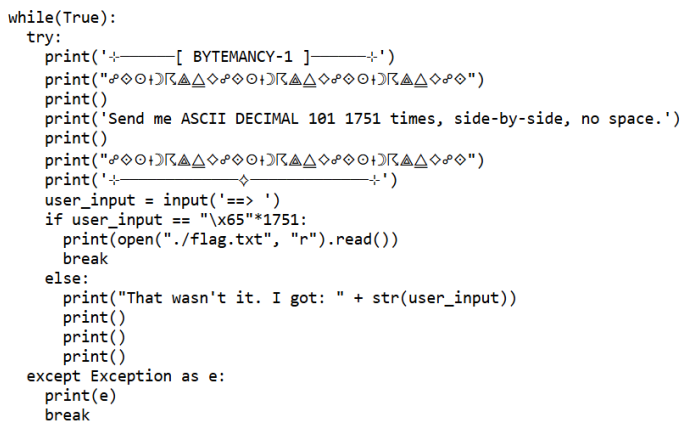
In the **bytemancy 1** challenge, I was given a task where I needed to connect to a remote server using a netcat command and send the correct input to retrieve the hidden flag. Unlike the previous bytemancy 0 challenge, where I only needed to enter three letters, this challenge required me to send the same character **1,751 times without spaces**. Because entering 1,751 characters manually would take a very long time and could easily lead to mistakes, I needed to find a way to automatically generate the correct input and send it to the server.

---

## 2. Initial Analysis / Reconnaissance

*What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?*

First, I opened the challenge and read the instructions and hint. I noticed that the challenge provided a source code file, so I downloaded the app.py file directly from the challenge page. Just like in the previous bytemancy 0 challenge, I opened the Python source code using Google Chrome. When I looked at the code, I noticed that the program was asking for **ASCII decimal 101 repeated 1,751 times without spaces**. From the previous challenge, I already knew that ASCII decimal 101 represents the lowercase letter **e**. So, this means that the program expects the letter e to be repeated exactly **1,751 times**. I realized that manually typing e 1,751 times would take too long, and there was a high chance that I would type the wrong number of characters. Because of this, I decided to use Python in the Linux terminal to automatically generate the required characters just like what's being said in the hint.



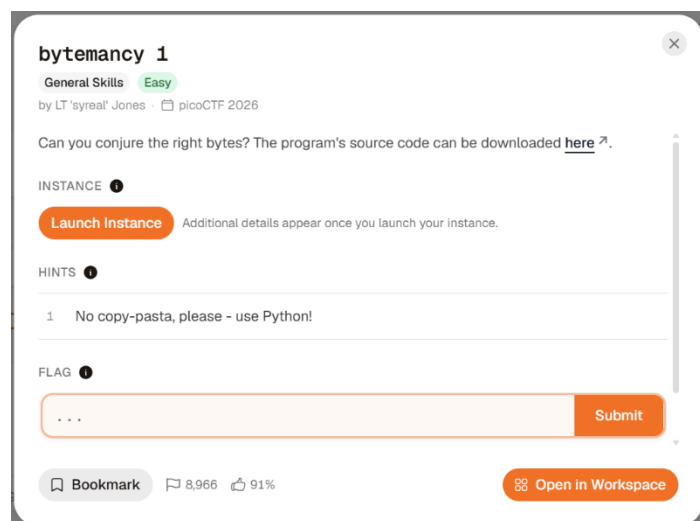
| Tool / Command                   | What it was used for                                         | Result / Output                                                                    |
|----------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------|
| Browser (Google Chrome)          | Downloaded and opened app.py to inspect the source code.     | Revealed that the server requires 1,751 repetitions of the letter e.               |
| nc foggy-cliff.picoctf.net 59664 | Connected to the remote challenge server                     | Opened the challenge connection in the terminal                                    |
| python3 -c                       | Automatically generated the required repeated character      | Generated the letter e 1,751 times                                                 |
| (Linux pipe)                     | Sent the output generated by Python directly to Netcat       | Passed the generated characters into the remote server                             |
| Gemini AI                        | Helped me find a way to automate the repeated terminal input | Suggested combining Python with Linux piping to send the generated input to Netcat |

## 4. Solution Walkthrough

*Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.*

## Step 1

The first step was to open the **bytemancy 1** challenge page and read the challenge description and hint. I noticed that there was a link that allowed me to download the source code. I clicked the link because I wanted to check how the program works and what kind of input it expected from me.



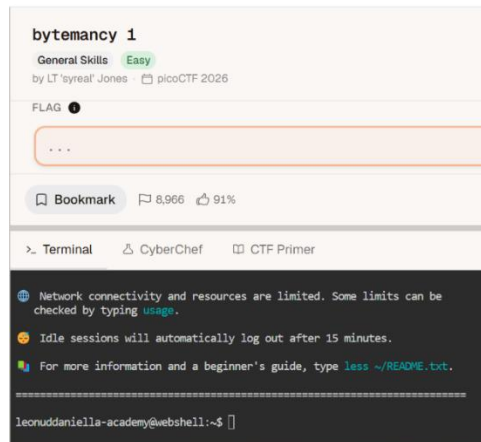
## Step 2

After downloading, I immediately checked what the given source code is.

```
while(True):
    try:
        print('÷————[ BYTEMANCY-1 ]————÷')
        print("⌘⊕⊕⊕⌚Ⓜ△△⊕Ⓢ⊕⊕⌚Ⓜ△△⊕Ⓢ⊕⊕⌚Ⓜ△△⊕Ⓢ⊕⊕")
        print()
        print('Send me ASCII DECIMAL 101 1751 times, side-by-side, no space.')
        print()
        print("⌘⊕⊕⊕⌚Ⓜ△△⊕Ⓢ⊕⊕⌚Ⓜ△△⊕Ⓢ⊕⊕⌚Ⓜ△△⊕Ⓢ⊕⊕")
        print('÷—————⊕—————÷')
        user_input = input('==> ')
        if user_input == "\x65"*1751:
            print(open("./flag.txt", "r").read())
            break
        else:
            print("That wasn't it. I got: " + str(user_input))
            print()
            print()
            print()
    except Exception as e:
        print(e)
        break
```

### Step 3

After understanding what the source code was asking for, I launched the challenge instance. The challenge gave me this netcat command **nc foggy-cliff.picoctf.net 59664**. I copied the command and opened the workspace.



### Step 4

Since typing 1,751 characters manually takes too long and introduces errors, I consulted **Gemini AI** for a way to automate terminal inputs. Based on the AI's guidance, I executed a Linux terminal one-liner that combines `python3 -c` with a pipe operator (`|`) to feed the generated string directly into the `nc` command:



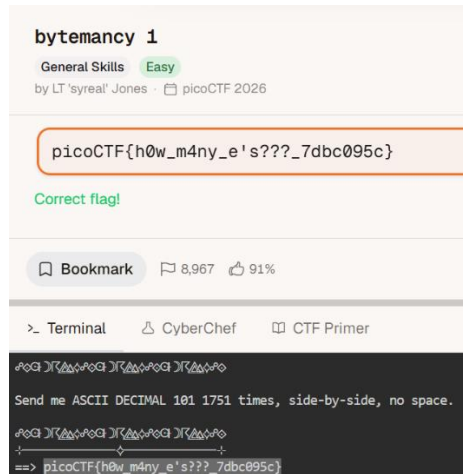
### Step 5

The remote server processes the 1,751 characters, validates the input, and returns the flag directly in the terminal output which is **picoCTF{h0w\_m4ny\_e's???\_7dbc095c}**



## Step 6

I copied the output flag, pasted it into the submission box, and then received the Correct flag notification



## 5. Flag

```
picoCTF{h0w_m4ny_e's???_7dbc095c}
```

## 6. Key Concept / What I Learned

*In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.*

This challenge taught me how to automatically generate and send repeated input using Linux terminal commands. I learned how Python can generate the same character a specific number of times and how the pipe symbol | can send the output of one command directly into another command such as netcat. Using automation was much easier and more accurate than manually typing the same character 1,751 times.

## 7. References

*List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.*

Gemini AI Assistance (<https://gemini.google.com/> )

Python Documentation (<https://docs.python.org/3/using/cmdline.html> )

GNU Netcat Documentation ([https://www.gnu.org/software/inetutils/manual/html\\_node/netcat-invocation.html](https://www.gnu.org/software/inetutils/manual/html_node/netcat-invocation.html))